

Programación Java



**Si tengo una duda y no pregunto por no pasar
vergüenza 5 minutos...
es muy probable que esa duda me haga pasar
vergüenza muchas veces en mi vida**

**POR FAVOR, preguntad todo lo que se os ocurra...
AQUÍ NOS HEMOS REUNIDO PARA APRENDER!!!**

Un poco de historia

1954-1957  FORTRAN

1959: Grace Hopper  COBOL

1972: Dennis Ritchie  C programming language

1986: Bjarne Stroustrup  C++

1995: Sun Microsystems  Java (primer versión oficial)

2002: Microsoft  C#

2010: Oracle Corporation  Compra Sun Microsystems

Orígenes de Java

1990-1992 “Green Project”  Oak ... Green

WWW de texto a gráfico  Distribución libre de Green

Primera aplicación  Browser para HTML

Green  Java

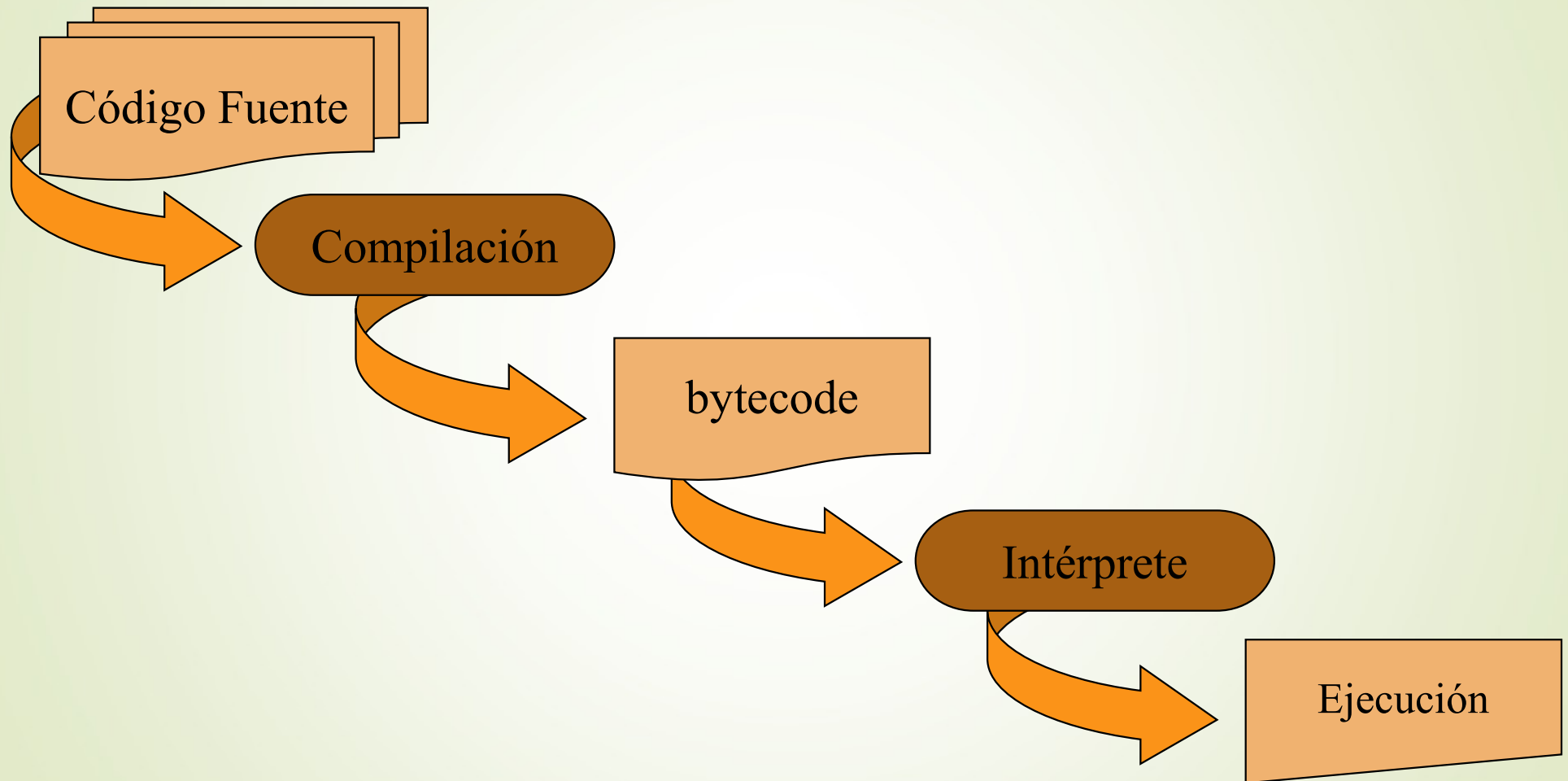
Versiones

año	Plataforma	Nombre	Características principales
1995	(Beta)		
1996	JDK 1.0		
1997	JDK 1.1		Inner classes, Java Beans, Reflection
1998	J2SE 1.2	Playground	Swing classes (GUI)
2000	J2SE 1.3	Kestrel	Java Naming and Directory Interface (JNDI)
2002	J2SE 1.4	Merlin	New I/O, regular expressions, XML parsing
2004	J2SE 5.0	Tiger	Generic types, annotations, enum types, ...
2006	Java SE 6	Mustang	Scripting language support
2011	Java SE 7	Dolphin	String in switch statement, exceptions, ...
2014	Java SE 8		API Fechas, métodos default en Interfaces, Lambda expressions
2017	Java SE 9	...	
2021	Java SE 17		

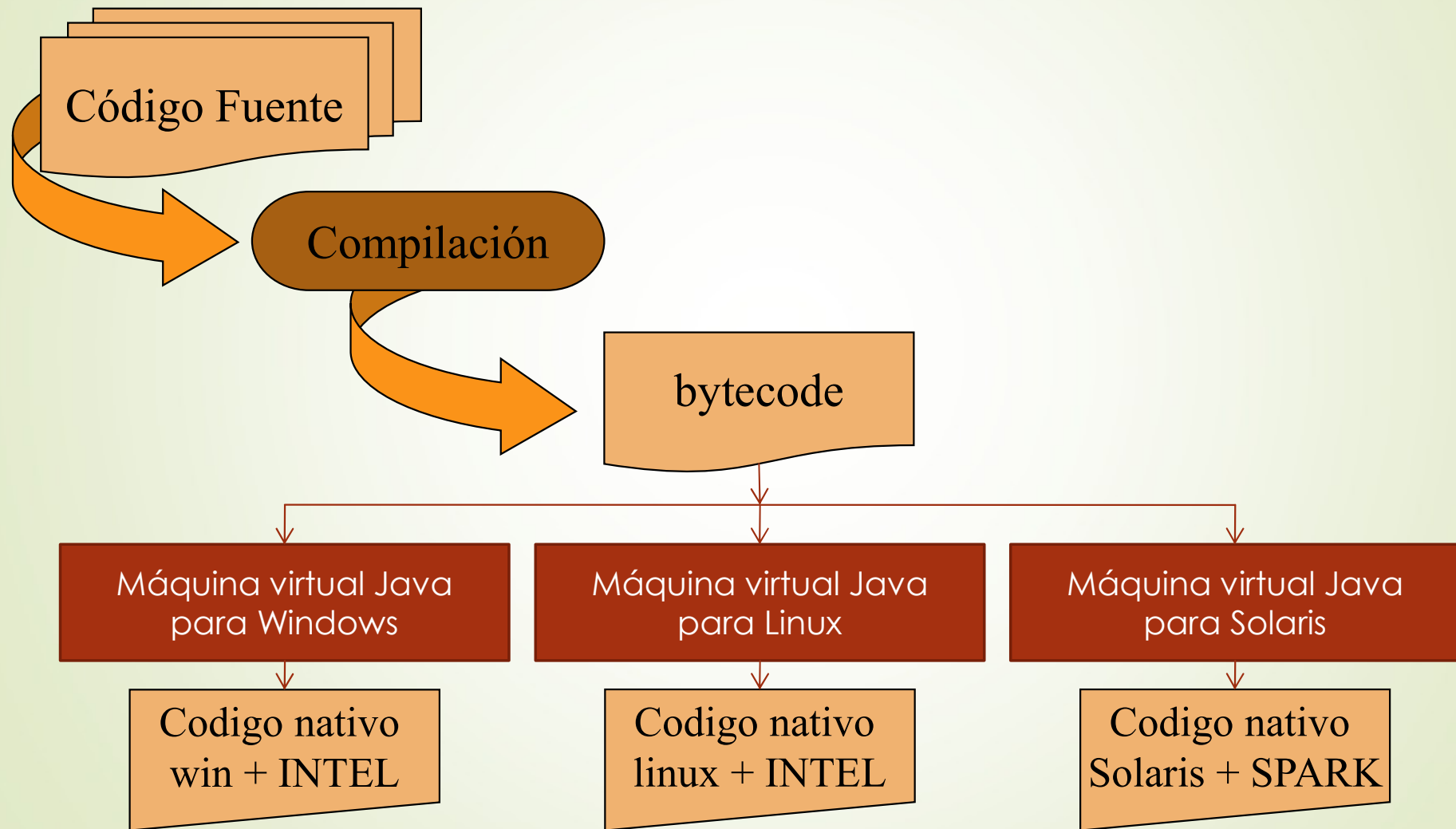
Características

- Lenguaje de propósito general.
- Sintaxis basada en C++.
- Orientado a objetos.
- Librerías: red, hilos, excepciones, gráficos.
- Código fuente y compilado, independiente de la plataforma.

Independencia de la Plataforma



Independencia de la Plataforma



JRE (Java Runtime Environment)

- Máquina virtual (java.exe)
- Conjunto de librerías estándar

JDK (Java Development Kit)

- Compilador (javac.exe)
- Intérprete (java.exe)
- Depurador (jdb.exe)
- Utilidades (jar, javadoc, etc.)
- Opcional: documentación y fuentes de librerías

Tecnología Java

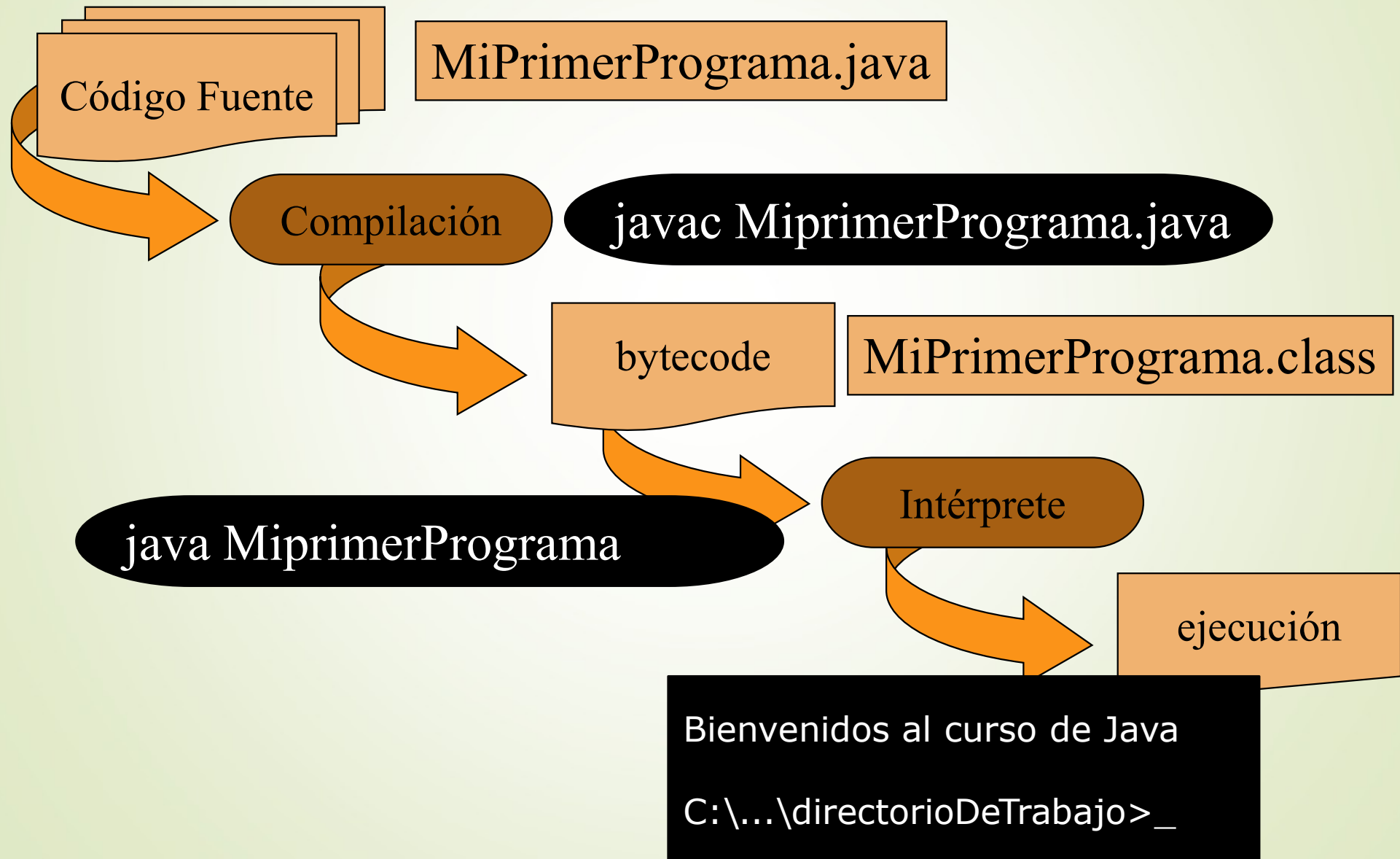
- Java SE (Java Platform Standard Edition)
- Java EE (Java Platform Enterprise Edition)
 - EJB (Enterprise JavaBeans)
 - Servlets y JSP
 - JPA
 - JSF

El Primer Programa

```
class MiPrimerPrograma {  
    public static void main(String[] arg) {  
        System.out.println("Bienvenidos al curso de Java");  
    }  
}
```

```
C:\...\directorioDeTrabajo>javac MiPrimerPrograma.java  
C:\...\directorioDeTrabajo>java MiPrimerPrograma  
Bienvenidos al curso de Java  
  
C:\...\directorioDeTrabajo>_
```

Compilación y Ejecución



Sintaxis

EXPRESIONES:

Conjunto de elementos o tokens que son evaluados para devolver un resultado o realizar una tarea.

TOKENS:

- **Identificadores:** nombre de variables, clases, paquetes, métodos y constantes.
- **Palabras claves:** Identificadores reservados del lenguaje
- **Literales:** valores que pueden tomar las variables, son valores constantes.
- **Operadores:** Indican la evaluación que se debe aplicar a un objeto o dato.
- **Separadores:** Indican al compilador donde se ubican los diferentes elementos del código: {},:,;
- Los comentarios, saltos de línea, espacios vacíos y de tabulación, no se consideran tokens, ya que el compilador los elimina.

Sintaxis

EXPRESIONES:

Conjunto de elementos o tokens que son evaluados para devolver un resultado o realizar una tarea.

TOKENS:

- Identificadores: nombre de variables, clases, paquetes, métodos y constantes.
- Palabras clave: `//...`
- Literales: valores constantes.
- Operadores: `while (i == 0) {`
`resu = a + b;`
`i = 1;`
`//bucle que no sirve para nada`
`}`
- Separadores: `System.out.println(resu);`
`//...`
- Los comentarios, retornos de carro, espacios vacíos y de tabulación, no se consideran tokens, ya que el compilador los elimina.

Sintaxis

- En java se hace distinción entre mayúsculas y minúsculas.
- Las instrucciones de java terminan en punto y coma.
- Los bloques de código se delimitan mediante llaves `{ }`

Identificadores

p.ej. nombres de variables

- Secuencia de letras, dígitos y los caracteres _ y \$
- Debe comenzar con una letra, \$ ó _. Se recomienda que inicien con una letra.
- Letra: cualquier carácter Unicode que describa una letra: ñ, á, Π, ä, etc.
- No se puede utilizar palabras reservadas.

Ej.:

nombre, saldoActual, PORCENTAJE_IVA,
año, paísDeNacimiento, valor01

Convenciones de nombres

Precio sin impuestos



kebab-case (skewer-case)

precio-sin-impuestos



camelCase

precioSinImpuestos



PascalCase o UpperPascalCase

PrecioSinImpuestos



snake_case

precio_sin_impuestos



SCREAMING_SNAKE_CASE

PRECIO_SIN_IMPUESTOS

Convenciones y Recomendaciones

Clases: **PascalCase** `class PrimerEjemplo { ..... }`

Métodos y variables: **camelCase**

```
void mostrarMensaje(){  
    String mensajeAMostrar = “Hola, buenas tardes”;  
    System.out.println(mensajeAMostrar);  
}
```

Constantes: **SCREAMING_SNAKE_CASE** `TIPO_IVA`

Paquetes: **lowercase** `java.util, java.lang`

(Las palabras reservadas del lenguaje son todas en minúsculas)

Palabras claves

abstract	boolean	break	byte	case
catch	char	class	continue	default
do	double	else	extends	false
final	finally	float	for	if
implements	import	instanceof	int	interface
long	native	new	null	package
private	protected	public	return	short
static	super	switch	synchronized	this
throw	throws	transient	true	try
void	volatile	while	var	rest
byvalue	cast	const	future	generic
goto	inner	operator	outer	

Comentarios

Junto al código fuente de un programa, se pueden insertar comentarios. Pueden ser de una sola línea, en ese caso empiezan con dos barras de división (//) o pueden ser de varias líneas, en ese caso el comentario se encierra con barra y asterisco (/* comentario de varias líneas */).

```
//ESTE ES EL PRIMER PROGRAMA

public class MiPrimerPrograma {
    /*muestra un mensaje
       por la
       consola
    */
    public static void main(String[] args) {//otro comentario
        System.out.println("Bienvenidos al curso de Java");
    }
}
```

Tipos Primitivos de Datos

		TIPO	TAMAÑO	RANGO
N U M É R I C O S	Enteros	byte	8 bits (1byte)	-128 a 127
		short	16 bits (2 bytes)	-32768 a 32767
		int	32 bits (4 bytes)	$-2... \times 10^9$ a $2... \times 10^9$
		long	64 bits (8 bytes)	$-9.. \times 10^{18}$ a $9.. \times 10^{18}$
	Reales	float	32 bits (4 bytes)	IEEE 754
		double	64 bits (8 bytes)	IEEE 754
		char	16 bits (2 bytes)	'\u0000' a '\uFFFF'
		boolean		true o false

Operadores

Aritméticos	$+$, $-$, $*$, $/$, $\%$
	$++$, $--$
Relacionales	$>$, $>=$, $<$, $<=$, $==$, $!=$
Lógicos	$\&\&$, $ $, $!$, $\&$, $ $
Desplazamiento y Lógicos de bits	$<<$, $>>$, $>>>$
	$\&$, $ $, $^$, $\sim$
Asignación	$=$, $+=$, $-=$, $*=$, $/=$, $\%=$
	$\&=$, $ =$, $^=$, $<<=$, $>>=$, $>>>=$
Condicional	$?:$

Ingreso por teclado

```
import java.util.Scanner;
```

```
public class A01IngresoTeclado {  
    public static void main(String[] args) {
```

```
        Scanner teclado = new Scanner(System.in);  
        int num;
```

```
        System.out.print("Introduzca un número entero: ");  
        num = teclado.nextInt();
```

```
        System.out.println("El número introducido es: " + num);
```

```
    }  
}
```



Comprobación De Tipos Estricta

<tipoDeDato> <identificador>;

Ej. 1:

```
int edad; //declaración  
edad = 32; //inicialización  
float altura, peso;
```



Ej. 2:

```
int edad = 32;  
int cont = 0, acu = 0;
```



Ej. 3:

✗
int a = 6;
b = a;

b no está
declarada

Ej. 4:

```
int cantidad = 45;  
int cantidad = cantidad + 1; ✗
```

cantidad ya
se declaró

Declaración de Constantes

```
final <tipoDeDato> <identificador>;
```

```
final int TIPO_IVA = 21;
```

```
final double COMISION = 1.5;
```

El Tipo char

“D” es una
cadena

```
char c1 = 'D';  
char c2 = 'a';
```

c2 es un
char

✗

```
char c3 = "D";
```


✗

```
String c4 = 'a';
```

‘a’ es un char, no
asimilable a
cadenas

```
String str = c2;
```

 ✗

```
String str = c2 + "";
```

 ✓

```
char c5 = 65;           // 'A'  
char c6 = 'A' + 1;      // 'B'  
int val1 = 'A';          // 65  
int val2 = 'A' + 'B';    // 131
```

 ✓

String: vista rápida

Los String en Java NO SON TIPOS PRIMITIVOS DE DATOS.

String es una clase de librería Java + operador de concatenación

No se deben comparar con ==, debe utilizarse el método .equals(..)

Declaración:

```
String palabra;  
palabra = "hola";
```

```
String nombre = "Pepe";  
String apellido = "Jimenez";
```

Ejemplo uso:

```
String nombreCompleto;  
nombreCompleto = apellido + ", " + nombre;  
System.out.println(nombreCompleto);
```

Literales(numéricos)

ENTEROS

LITERAL	BASE	VALOR
101	Decimal (10)	101
0b101	Binario (2)	5
0101	Octal (8)	65
0x101	Hexadecimal (16)	257

Ej: 5 -578 +5 12_000_000 12_00

REALES

LITERAL	NOTACIÓN	VALOR
15300.0	Normal	15.300,0
1.53e+4	Científica	15.300,0

Ej: .5 -.5 +.5 .7E10 .3e-10

Literales

CARACTERES

LITERAL	VALOR
'A'	A
65	A
'\u0041'	A

BOOLEAN

LITERAL	VALOR
true	true
false	false

STRING

“No soy exactamente un literal, sino un objeto de la clase String”

Literales y tipos

- Además de su valor, tienen TIPO.
 - En java $5 \neq 5.0$
- Literales enteros: su tipo es int pero pueden asignarse valores a byte y short que serán convertidos automáticamente.
- Literales reales: su tipo de datos es double.

SUFIJOS

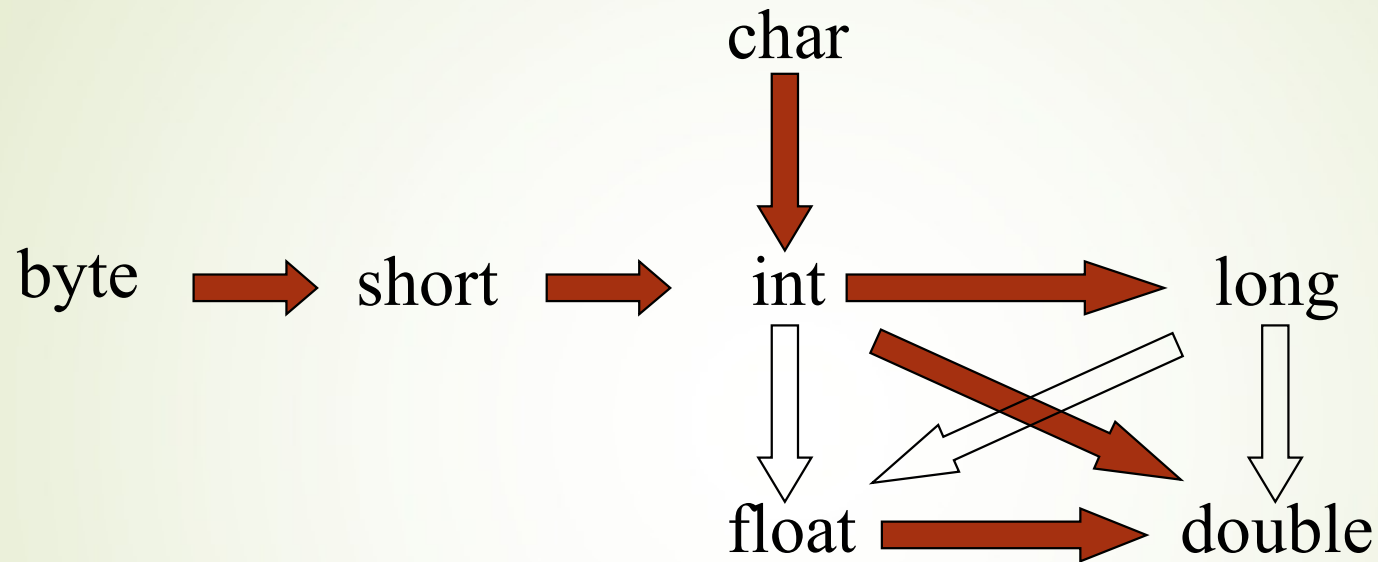
```
byte b = 98;  
short s = 32000;  
int i = 100_000;
```

```
float val = 2.5F;  
long l = 3_000_000_000L;
```

Secuencias De Escape

<code>\"</code>	<code>"</code>
<code>\'</code>	<code>'</code>
<code>\b</code>	Carácter backspace (BS)
<code>\t</code>	Tabulador (TAB)
<code>\n</code>	Nueva Línea (LF)
<code>\r</code>	Retorno de Carro (CR)
<code>\f</code>	Avance de Página (FF)
<code>\uXXXX</code>	Carácter UNICODE, XXXX hexadecimal
<code>\\</code>	<code>\</code>

Compatibilidad De Tipos De Datos



```
byte b = 65;  
int i = b;  
double d = i;
```



```
int a = 12.0; ❌
```

```
int peq = 8;  
byte b1 = peq; ❌
```

12.0 es
double

peq es int

Conversiones Explícitas (typecast)

(tipo_de_dato) expresión

```
float sueldoAnual;
```

```
...
```

```
 int sueldoMensual = sueldoAnual / 12;
```

sueldoAnual es
float, la expresión es
float

```
float sueldoAnual;
```

```
...
```

```
int sueldoMensual = (int) (sueldoAnual / 12);
```



Sentencias

- Simples
- Compuestas

- Toda sentencia simple finaliza con ;
- Sentencia compuesta o bloque: conjunto de sentencias, se delimita con {}

```
//Calculo raices
{
    discr = Math.sqrt(b * b - 4 * a * c);
    if(discr >= 0) {
        x1 = (- b + discr) / (2*a);
        x2 = (- b - discr) / (2*a);
    }
}
```

Estructuras De Control

Condicional Simple

```
if (expresión_lógica)  
    sentencia;
```

Condicional Doble

```
if (expresión_lógica)  
    sentencia_1;  
else  
    sentencia_2;
```

Estructuras De Control (cont.)

Condicional Múltiple

```
switch (pivote) {  
    case valor_1:  
        sentencia_1;  
    case valor_2:  
        sentencia_2;  
    ...  
    case valor_n:  
        sentencia_n;  
    default:  
        sentencia;  
}
```

El pivote puede ser:

- char
- byte
- int
- enum
- String (desde JDK7)

Estructuras De Control (cont.)

Condicional Múltiple

```
SW: int dia;  
    // ...  
    switch (dia) {  
    case 1: System.out.println("Lunes");  
           break;  
    case 2: System.out.println("Martes");  
           break;  
    case 3: System.out.println("Miércoles");  
           break;  
           // ...  
    case 7: System.out.println("Domingo");  
           break;  
    default: System.out.println("Día incorrecto");  
    }  
}
```

Estructuras De Control (cont.)

Estructuras Iterativas

```
while (expresión_lógica)  
    sentencia;
```

```
do  
    sentencia;  
while (expresión_lógica);
```

```
int i = 5;  
while (i < 8) {  
    System.out.println(i++);  
}
```

```
int i = 5;  
do {  
    System.out.println(i++);  
} while (i < 8);
```

Estructuras De Control (cont.)

Estructuras Iterativas

```
for (sentencia_inicializ; expresión_lógica; sentencia_actualiz)  
    sentencia;
```

```
for (int i = 1; i < 8; i++) {  
    System.out.println(i);  
}
```

```
int i;  
for (i = 1; i < 8; i++) {  
    System.out.println(i);  
}
```


Estructuras De Control (cont.)

Estructuras de Ruptura

➤ **break**

Rompe la ejecución de la estructura de control.
La ejecución continúa en la siguiente instrucción de la estructura.

➤ **continue**

Rompe la ejecución de la iteración actual de la estructura de control.
La ejecución continúa en la siguiente iteración.

➤ **return**

Sale de la rutina actual y devuelve el control al llamador.
Devuelve el valor necesario en funciones.

Métodos

```
[especificadores] tipoDevuelto nombreMetodo([lista parámetros]) [...] {  
    // instrucciones  
    [return valor;]  
}
```

tipoDevuelto

- Tipos Primitivos
- Objetos
- void

Lista parámetros

- Tipo nombre
- Separados por ,

return valor;

- Obligatorio si TipoDevuelto no es void
- El tipo de valor debe ser compatible con tipoDevuelto

Métodos

```
public class ClasePrueba {  
    public static void mensaje(){  
        System.out.println("Ejecutado el procedimiento");  
    }  
  
    public static long suma(int a, int b){  
        long resu;  
        resu = a + b;  
        return resu;  
    }  
  
    public static long producto(int a, int b){  
        return a * b;  
    }  
  
    public static void main(String[] args) {  
        mensaje();  
        long resultado;  
        resultado = suma (17, 89);  
        System.out.println("17 + 89 = " + resultado);  
        int a = 20, b = 35;  
        System.out.println(a + " * " + b + " = "+ producto(a,b));  
    }  
}
```

Sobrecarga de métodos

Signature

```
public class ClasePrueba {  
    public static long producto(int a, int b){  
        return a * b;  
    }  
}
```

`producto(int , int )`

`paquete.ClasePrueba.producto(int , int )`

Sobrecarga de métodos

- No pueden existir dos métodos con la misma “signature”
- Pueden existir varios métodos con el mismo nombre, siempre y cuando difiera la lista de parámetros.

```
public class ClasePrueba {  
  
    public static long producto(int a, int b){  
        return a * b;  
    }  
  
    public static double producto(double a, double b){  
        return a * b;  
    }  
}
```

Ambito Variables (scope)

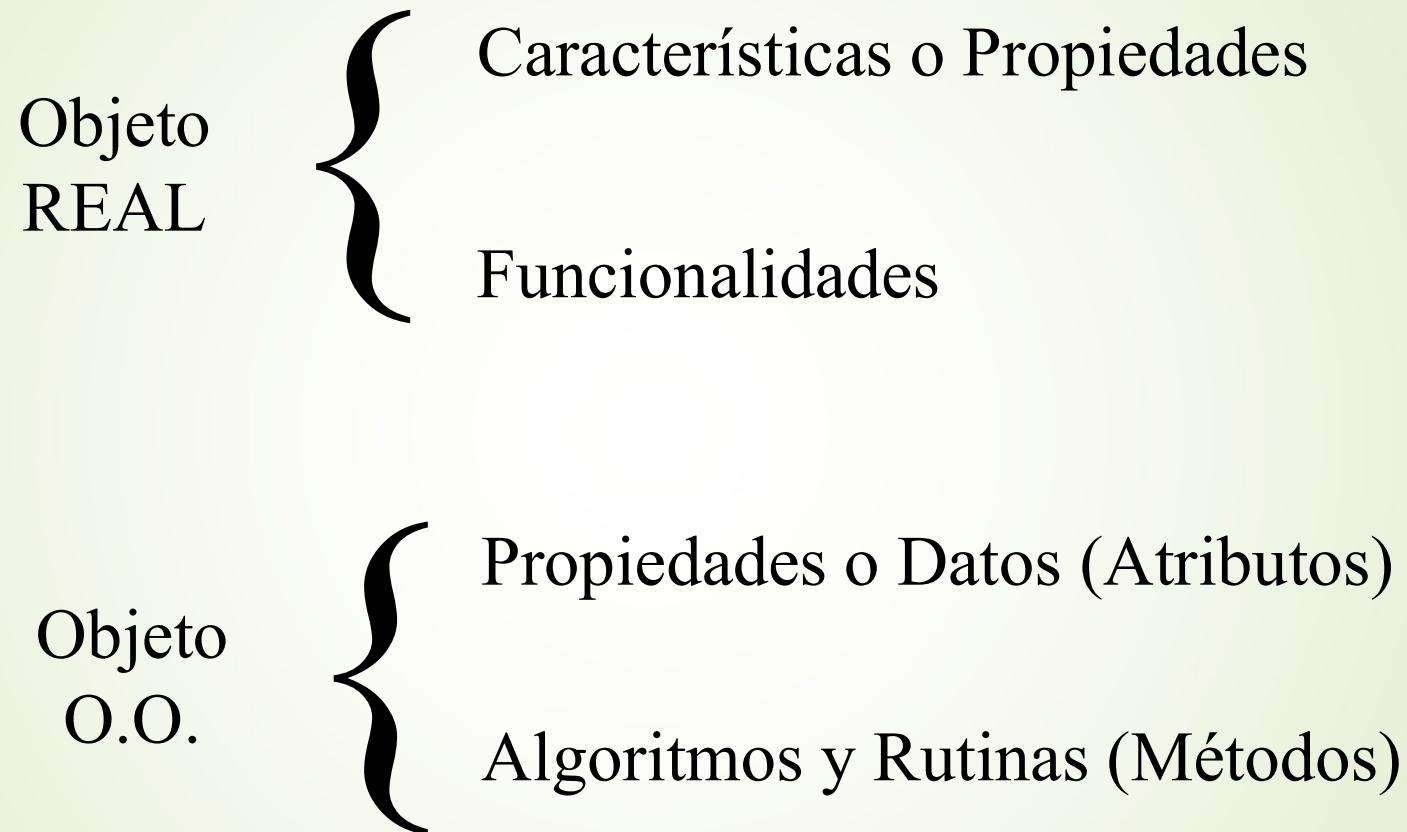
```
public class AmbitoVariables {
    static int atributoDeClase = 1;

    public static void main(String[] args) {
        int localMain = 2;
        while (localMain == 2) {
            int localWhile = 3;
            System.out.println(atributoDeClase);
            System.out.println(localMain);
            System.out.println(localWhile);
        }
        // System.out.println(localWhile);
        System.out.println(localMain);
        System.out.println(atributoDeClase);
        {
            int localBloque = 4;
            System.out.println(localBloque);
        }
        // System.out.println(localBloque);
    }

    public static void otroMetodo(){
        System.out.println(atributoDeClase);
        // System.out.println(localMain);
    }
}
```

Orientación a Objetos

Objeto



Orientación a Objetos

- **Objetivo principal:**

Reducir la complejidad del desarrollo y mantenimiento de sw.

- **Ventajas:**

Facilita el desarrollo de sistemas complejos.

Facilita la reutilización.

Características De Un Objeto

- Identidad

Es la propiedad que permite **diferenciar a un objeto y distinguirse de otros.**
- Estado

Se refiere al conjunto de los valores de sus atributos en un instante de tiempo dado. El comportamiento de un objeto puede modificar el estado de este.
- Comportamiento

Está directamente relacionado con su **funcionalidad** y determina las **operaciones que este puede realizar.**

Paradigmas De La O.O.

- Abstracción

- Encapsulamiento

- Herencia

- Relaciones entre objetos

 - Asociación o uso (utiliza un...)

 - Agregación y composición (tiene un...)

 - Herencia (es un...)

 - Mensajes

- Polimorfismo

Abstracción



Abstracción



Es el proceso de simplificar un problema complejo enfocándose tan sólo en los aspectos relevantes para la solución. En el desarrollo de software esto significa centrarse en lo que es y hacer un objeto antes de decidir cómo debería ser implementado.

TDA
Tipo de **D**ato **A**bstracto

Paradigmas De La O.O.

- Abstracción

- Encapsulamiento

- Herencia

- Relaciones entre objetos

 - Asociación o uso (utiliza un...)

 - Agregación y composición (tiene un...)

 - Herencia (es un...)

 - Mensajes

- Polimorfismo

Encapsulamiento



La POO permite dividir un programa en componentes más pequeños e independientes. Cada componente es autónomo y realiza su labor independientemente de los demás componentes. El encapsulamiento mantiene esta independencia **ocultando los detalles internos** (la implementación) de cada componente, mediante una interfaz externa.

Paradigmas De La O.O.

- Abstracción

- Encapsulamiento

- Herencia

- Relaciones entre objetos

 - Asociación o uso (utiliza un...)

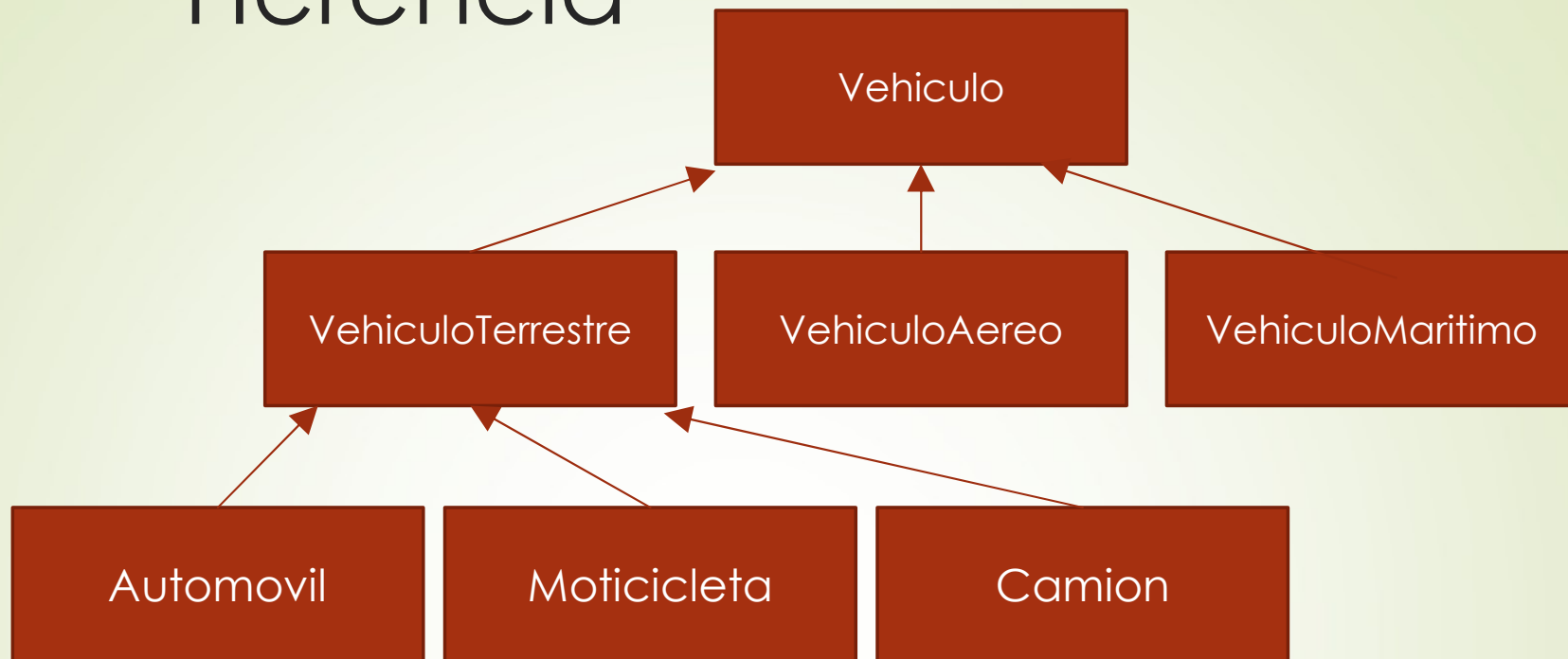
 - Agregación y composición (tiene un...)

 - Herencia (es un...)

 - Mensajes

- Polimorfismo

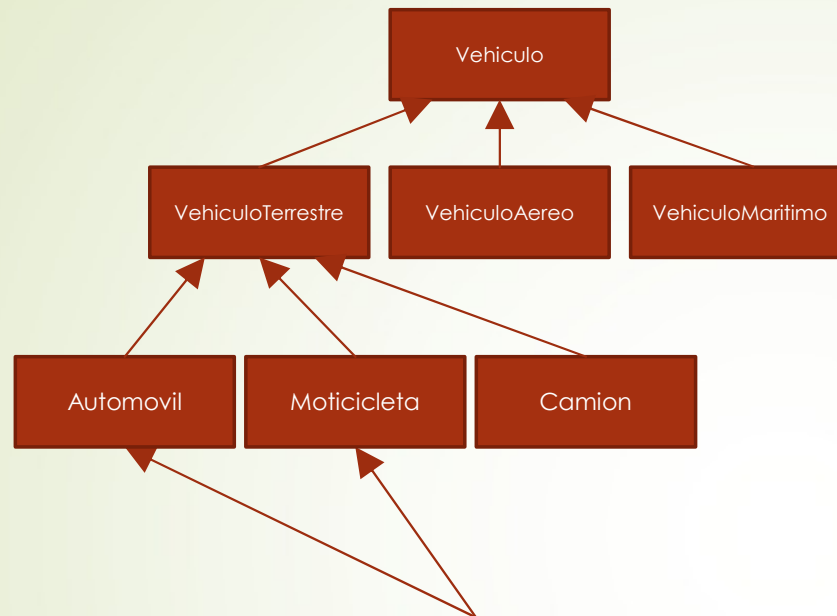
Herencia



La Herencia permite agrupar propiedades y funcionalidades en común de varias clases en una nueva, creando una estructura jerárquica de clases cada vez más especializada.

Se pueden adquirir bibliotecas de clases que ofrecen una base que puede especializarse a voluntad (la compañía que vende estas clases tiende a (y debe) proteger la implementación usando la encapsulación).

Herencia

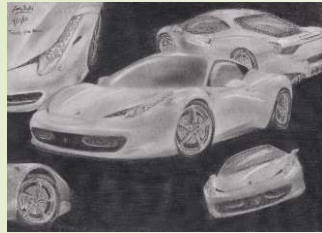


SIMPLE



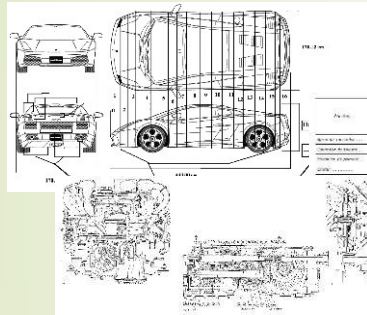
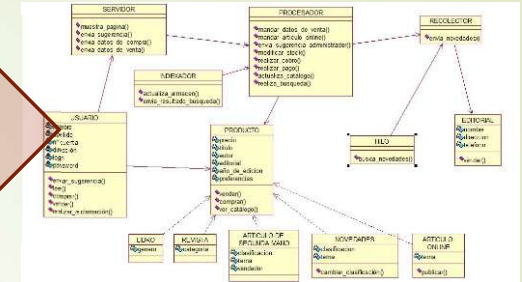
MÚLTIPLE

Clase y Objeto



Diseño (concepto)

Diseño UML



Diseño detallado
(especificaciones)

Clase Java

```
using System;
using NUnit.Framework;

namespace TestCodeCoverage
{
    public class Test<T>
    {
        where T: struct
        {
            public void TestMethod(T x) {
                Console.WriteLine(x);
            }
        }
    }

    [TestFixture]
    public class TestFixture
    {
        [Test]
        public void DoTest()
        {
            var t = new Test<bool>();
            t.TestMethod(true);
        }
    }
}
```



Fabricación

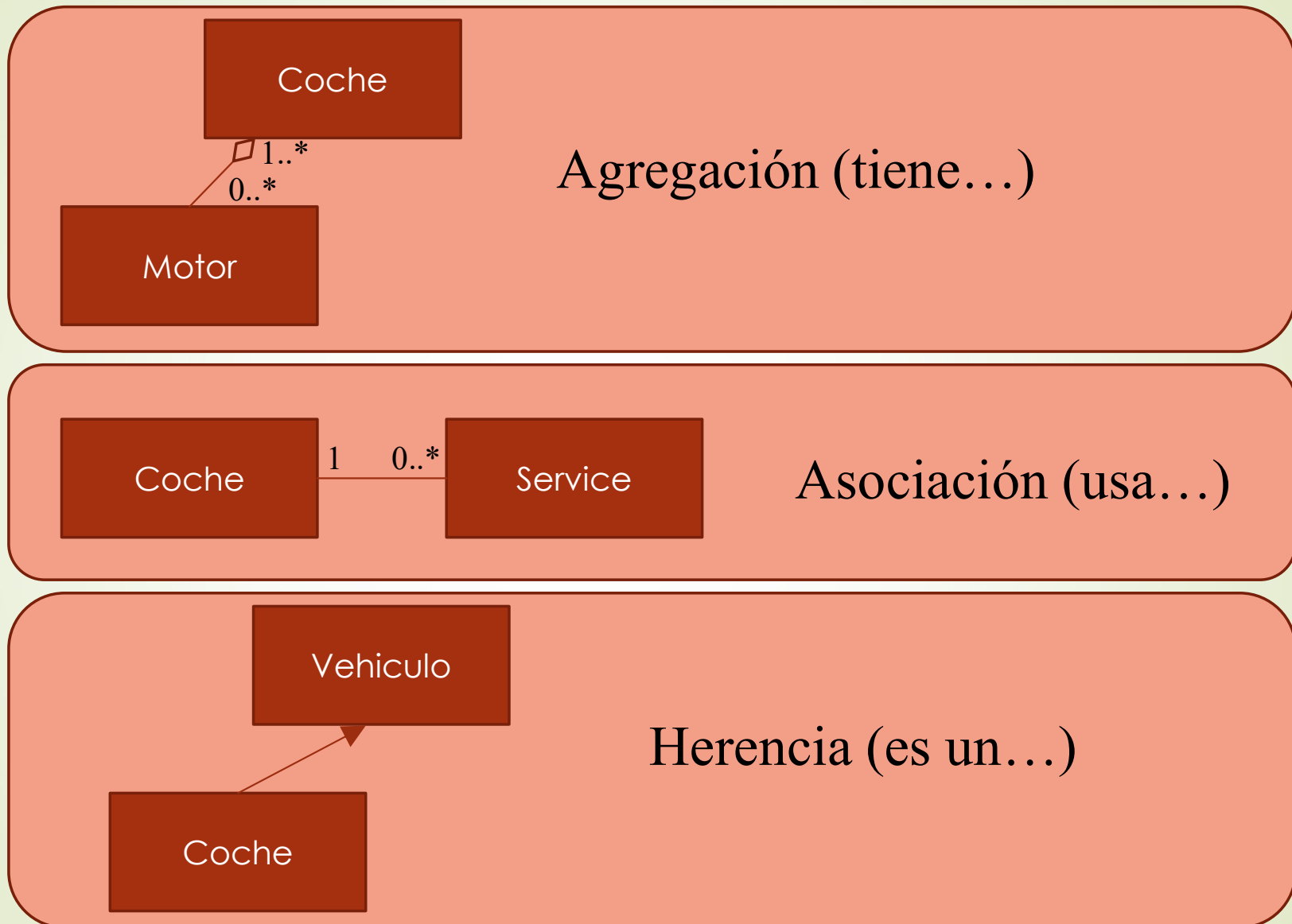
Objeto Java (instancia)



Paradigmas De La O.O.

- Abstracción
- Encapsulamiento
- Herencia
- Relaciones entre objetos
 - Asociación o uso (utiliza un...)
 - Agregación y composición (tiene un...)
 - Herencia (es un...)
 - Mensajes
- Polimorfismo

Relaciones entre objetos



Atributos y Métodos

➤ Estaticos

Conservan el mismo valor en todas los objetos.

Es como una constante de clase

➤ No Estáticos

Métodos static

Los métodos estáticos solamente pueden acceder a los atributos estáticos de la clase, no a los atributos dinámicos porque no sabrían de que objeto tomarlo.

Ejemplo de una clase con un método estático que facilita la tarea de leer del teclado.

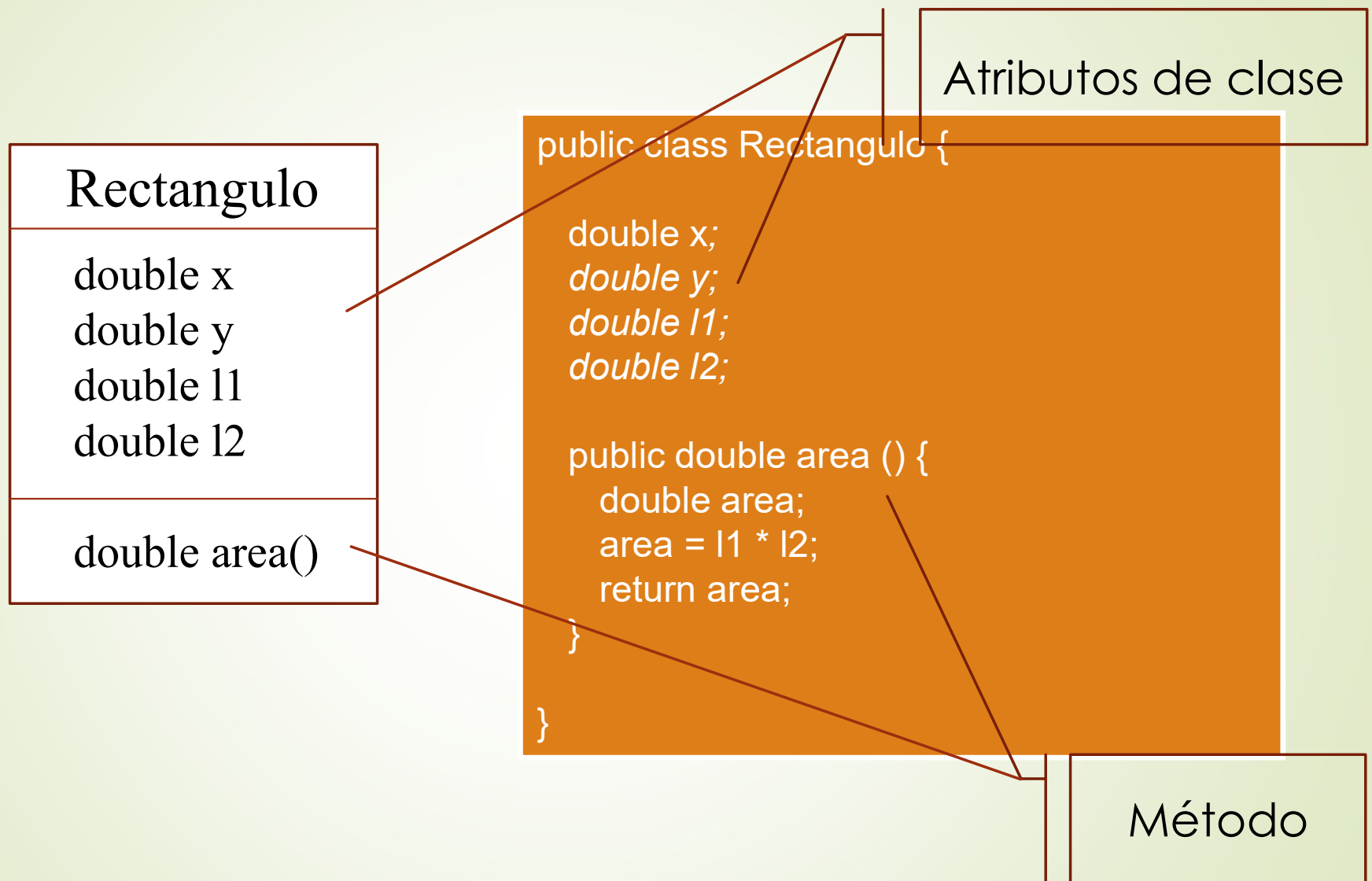
```
import java.util.Scanner;

public class Teclado{
    static Scanner teclado = new Scanner(System.in);

    public static String leer(){
        return teclado.nextLine();
    }
}
```

Uso del método estático ejecutándolo directamente desde la clase, sin necesidad de una instancia.

Miembros de Clase



Métodos

- Modificador (setter)
- Selector (getter)
- Iterador
- Constructor
- Destructor
- Propósito general

Clases Básicas Del API

Clase Math

La clase Math proporciona métodos static (que se ejecutan sobre la clase sin necesidad de crear instancias) para realizar las operaciones matemáticas más habituales.

abs()	valor absoluto	sqrt()	raiz cuadrada.
cos(), acos()	coseno y arco coseno	pow(,)	el primer argumento elevado al segundo
tan(), atan(), atan2(,)	tangente, arco tangente entre $-\pi/2$ y $\pi/2$, arco tangente entre $-\pi$ y π .	exp(), log() log10()	el número “e” elevado al argumento. Logaritmo neperiano Logaritmo decimal.
sin(), asin()	seno y arco seno	random()	número aleatorio entre [0 y 1[
ceil(), floor()	redondeo por exceso y por defecto.	toDegrees()	convierte radianes a grados
round()	entero más cercano	toRadians()	convierte grados a radianes
E	constante, número e	PI	constante, número π

Clase String

Un String es un Objeto. Se debe instanciar la clase String.

Para acceder a un objeto, también se emplean variables (hacer referencia o apuntar) a los objetos que haya creados en la memoria del sistema.

Declaración:

```
String miTexto; // igual que con tipos primitivos
```

Creación de un objeto y asignación a una variable:

```
miTexto = new String();           // Cadena vacía.
```

```
miTexto = new String("Hola");     // Con un texto.
```

```
miTexto = "Hola";                 // objeto String abreviado
```

Asignación de referencia a un objeto desde otra variable:

```
String otroObjeto = miTexto;
```

Asignación de "ninguna" referencia:

```
miTexto = null;
```

Clase String (II)

Ejemplo: `String s = “Hola a todos”;`

Métodos

`int length()`

ej. `int tam = s.length(); // tam = 12`

Devuelve el número de caracteres de la cadena

`char charAt(int indice)`

ej. `char letra = s.charAt(3); // letra = ‘a’`

Devuelve el carácter en la posición especificada por índice.

`int indexOf(String str)`

ej. `int pos = s.indexOf(“tod”); //pos = 7`

Devuelve la posición en la que aparece por primera vez un String (str) en el string sobre el que se aplica el método. Si no la encuentra retorna -1.

`String substring(int beginIndex, int endIndex)`

ej. `String sub = s.substring(7, 11); //sub = “todo”`

Devuelve un String extraído a partir de beginIndex y hasta endIndex.(sin incluirlo)

Clase String (III)

Ejemplo: String s = “Hola a todos”;

Métodos

boolean **startsWith**(String prefix) **ej. s.startsWith(“Hola”); //true**

Indica si un String comienza con otro String o no.

String **toLowerCase**() **ej. String min = s.toLowerCase(); //min=“hola a todos”**

Retorna una nueva cadena convertida a minúsculas.

String **toUpperCase**() **ej. String may = s.toUpperCase(); //min=“HOLA A TODOS”**

Retorna una nueva cadena convertida a mayúsculas.

String **replace**(String oldStr, String newStr)

ej. String s2=s.replace(“todos”,”todas”); // s2=“Hola a todas”

Retorna un String en el que se ha sustituido una subcadena (oldStr) por otra (newStr).

StringBuffer y StringBuilder

Principales métodos

append(...)	agrega distintos tipos de datos convertidos a String al final de la cadena
insert(...)	agrega distintos tipos de datos convertidos a String en la posición indicada
deleteCharAt(int index)	Borra el carácter indicado
delete(int start, int end)	Borra los caracteres desde hasta(s/incl.)
indexOf(...)	
charAt(int index)	
replace(...)	
Substring(...)	

Classes Wrapper

byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

```
int edad = Integer.parseInt("30");  
double importe = Double.parseDouble("15.45");  
  
int Integer.parseInt(String valor, int base)  
String Integer.toBinaryString(int valor)  
String Integer.toHexString(int valor)  
String Integer.toOctalString(int valor)  
String Integer.toString(int valor, int base)
```


Arrays

Arrays (I)

Son objetos con una característica especial, que pueden contener referencias, ordenadas bajo un índice, a otros objetos.

Definición de matrices de tipos primitivos de datos:

```
double[ ] x = new double[100]; // también double x[]
```

Definición de matrices de referencias a objetos:

```
String[ ] s = new String[3];
```

Acceso a elementos de una matriz:

```
x[2]=5.3;
```

```
s[0]= new String("Martin"); s[0] = "Maritn";
```

Inicialización:

```
int v[ ] = {4, 1, 6, 3, 42, 51, 61, 27, 98, 119};
```

```
String [ ] dias ={"lunes", "martes"};
```

```
System.out.println("Hoy es "+ dias[1]); // "Hoy es martes"
```

Arrays (II)

- Los índices de una matriz van desde cero hasta el tamaño menos uno.
- El tamaño se puede conocer con la propiedad length.

Ejemplo de bucle que recorre todos los elementos de una matriz:

```
int [] matriz ={53, 4, 43, 54};  
for( int i=0; i<matriz.length; i++) {  
    System.out.println(matriz[i]);  
}
```

- Se pueden crear matrices anónimas (por ejemplo: como parámetro de un metodo.

```
obj.metodo( new String[ ] {"uno", "dos", "tres"} );
```

Arrays (III)

Una matriz bidimensional es un vector de referencias a los vectores fila. Así pues, cada fila podría tener un número de elementos diferente.

Una matriz se puede crear directamente en la forma:

```
int [ ][ ] mat = new int[3][4];
```

O dando los siguientes pasos:

1. Crear la referencia (con un doble corchete): `int [ ][ ] mat;`
2. Crear el vector de referencias a las filas:

```
mat = new int[3][ ];
```

3. Crear los vectores correspondientes a las filas:

```
for (int i=0; i<3; i++) mat[i] = new int[4];
```

4. Asignar valores a cada elemento

```
for (int i=0; i<3; i++) for (int j=0; j<4; j++)  
    mat[i][j] = 2*i*j;
```